

The Dock Lab Kit

```
$ python -c "import e1_kaveri_mcp"          # silence
$ python e4_chunking.py
fixed    :    8 chunks | full procedure in one chunk: False
heading  :   10 chunks | full procedure in one chunk: True
$ python e7_dock_cameras.py | tail -4
retrieve mean=    399ms p95=  2,597ms  <- the mean lied
```

Every exhibit in the Dock, hand-typeable and runnable, with the files the Dock references but never ships: SOP 214 with nine real headed sections, the nine-tool belt, the scope table, a 10,000-ticket log day with a cold-carrier tail baked in, and DRY_RUN stubs so nothing costs money until you want it to. Each lab: predict, run, compare, sabotage one variable, fix it back, ledger. Every expected output below was produced by actually running the kit.

THE DOCK LAB KIT · HOW TO USE IT

Setup, rules, and the one honesty note

Unzip kaveri-dock-lab.zip into your kaveri-ops folder. Then, in PowerShell:

```
cd C:\Users\ismra\Documents\kaveri-ops\kaveri-dock-lab
python -m pip install fastmcp          # only E1 needs it (kit tested on fastmcp 4.0.10)
$env:DRY_RUN = "1"                    # free probes for E2 and E5; Remove-Item Env:DRY_RUN for live c
alls
python gen_logday.py                  # once: writes logday.jsonl, 40,000 lines
```

File	Exhibit	Needs
e1_kaveri_mcp.py · e1_probe.py	E1 the standard connector	fastmcp
e2_bloat_audit.py · tools_all9.json	E2 the bloat audit	DRY_RUN=1, or ANTHROPIC_API_KEY for the real count_tokens
e3_gate_pass.py	E3 the gate pass	nothing
e4_chunking.py · sops/sop_214.md	E4 chunking three ways	nothing
e5_hybrid.py	E5 the hybrid	DRY_RUN=1 (a toy foreman; see the honesty note)
e6_shape_router.py	E6 the retrieval router	nothing
gen_logday.py · e7_dock_cameras.py	E7 the cameras	nothing
e8_two_bills.py	E8 two bills	nothing

★ THE RULES

1. **Hand-type** the exhibit from the Dock first; open the kit's file only to diff against yours.
2. **Predict before run** — every lab below has a predict box; write the answer down before the command.
3. **Free probe before paid call** — DRY_RUN=1 is the bench; unset it only when you want the real count_tokens.
4. **Sabotage one variable**, predict, run, compare with the real result printed here, **fix it back**, ledger the surprise.

⇌ DRIFT · THE DOCS MOVED

What older teaching says: The Dock's expected outputs assume a ~4,000-word SOP 214, real count_tokens numbers (2,314 / 811), and print progressive = 4,625 tokens/call.

What the current docs say (checked Sep 2026): The kit's SOP 214 is ~1,330 words with nine headed sections, so E4's tape measure is **size=180** (8 boxes) to reproduce the same failure. E2 in DRY_RUN *estimates* tokens at ~4 chars each (726 / 256) — the live call is the number that counts. And E8's arithmetic: 225 + 260 + 3,600 + 120 = **4,205**, not 4,625 — the Dock's printed figure has a slip; the code is right, and the ratio is 86×, not 78×. Trust what runs.

Lab E1 · The standard connector — tools, a resource, a prompt

Objective 3.7 · Prove the import-probe guard holds, then list the three primitives with an in-process client (no stdio, no network).

Run

```
python -c "import e1_kaveri_mcp"      # expect: silence
python e1_probe.py
```

PREDICT FIRST — write it in the notebook

What prints from the import probe, and why does the server NOT start?

Which primitive would a client use to read SOP 214 without spending a tool call?

Will get_shipment_status's inputSchema say string or integer for consignment_id?

WHAT THE KIT ACTUALLY PRINTS

```
tools/list      -> ['get_shipment_status', 'find_alternate_carriers']
resources/list  -> ['sop://214']
prompts/list    -> ['customs_triage']
inputSchema of get_shipment_status -> {"type": "object", "additionalProperties": false,
    "properties": {"consignment_id": {"type": "string"}}, "required": ["consignment_id"]}
resources/read sop://214 -> first line: # SOP 214 – Customs holds and e-way bill re-filing
tools/call get_shipment_status -> {'id': 'KF-2481', 'status': 'CUSTOMS_HOLD', 'missing': 'e-way bill'}
```

© WHAT TO NOTICE

The schema was written by the type hint, the description by the docstring, the name by the function. resources/read cost no tool call. The probe used fastmcp's in-process Client — the same tools/list → tools/call shape a real client speaks, minus the transport.

Sabotage — break exactly one thing, predict, run, fix it back

Break this	Predict, then observe (real result)	Lesson
line 7: consignment_id: int	Nothing crashes. The schema silently becomes "type": "integer" — the model will now send numbers, and 'KF-2481' fails validation.	Type hints write the contract; a wrong hint is a wrong phone book.
delete the docstring on line 8	tools/list still shows the tool, with an empty description.	The docstring IS the interface; an empty card is a tool the model cannot choose well.
remove the __main__ guard	The import probe now STARTS the server and hangs waiting on stdio.	The guard is what keeps the free probe free.

Lab E2 · The bloat audit — measure the belt, then trim it

Objective 3.1 · Price two belts (tools_all9.json vs the three the logs use) and multiply by the day's traffic.

Run

```
$env:DRY_RUN = "1"; python e2_bloat_audit.py      # estimate
Remove-Item Env:DRY_RUN; python e2_bloat_audit.py  # live count_tokens (free API call, needs key)
```

PREDICT FIRST — write it in the notebook

Guess the token cost of ONE schema in tools_all9.json. Then the daily belt tax at 40,000 calls. Will the DRY_RUN estimate be higher or lower than the live count?

WHAT THE KIT ACTUALLY PRINTS

```
9 tools: 726 tokens / call
3 tools: 256 tokens / call
belt tax: 18,800,000 tokens/day for tools the logs say were called 0 times last week
# live count_tokens will print larger numbers (the Dock saw 2,314 / 811 -> 60,120,000)
```

🕒 WHAT TO NOTICE

Whatever the exact count, the shape is the point: ~6 idle schemas × N tokens × 40,000 calls. Open tools_all9.json and read lookup_order next to get_shipment_status — that is the lookalike pair from Q2.

Sabotage — break exactly one thing, predict, run, fix it back

Break this	Predict, then observe (real result)	Lesson
add escalate_to_human to CORE3	3-tool bill rises by ~80 tokens; belt tax falls slightly.	A load-bearing rare tool costs one schema a call — cheap insurance for the worst day of the month.
daily = 400_000	belt tax ×10 — 188M/day in DRY_RUN.	Bloat is linear in volume; the argument gets stronger, not weaker, at scale.
delete the description of lookup_order	Token count drops ~30; the wrong-tool problem gets WORSE.	Bloat has a confusion bill the token count cannot see.

Lab E3 · The gate pass — the scoped key, checked in code

Objective 3.2 · Three callers try process_refund; one of them holds a wildcard. Then the one-line fix.

Run

```
python e3_gate_pass.py
```

PREDICT FIRST — write it in the notebook

Outcome for kavi-agent, refund-desk and carrier-portal-v1 calling process_refund BEFORE the fix?

How many audit lines will print — only for allowed calls, or for denials too?

WHAT THE KIT ACTUALLY PRINTS

```
audit: {"caller": "kavi-agent", "tool": "process_refund", "decision": "DENY", "reason": "kavi-agent lacks refund:write for process_refund"}
kavi-agent      -> process_refund  PermissionError: kavi-agent lacks refund:write for process_refund
audit: {"caller": "refund-desk", "tool": "process_refund", "decision": "ALLOW"}
refund-desk     -> process_refund  OK {'refunded': 4200}
audit: {"caller": "carrier-portal-v1", "tool": "process_refund", "decision": "ALLOW"}
carrier-portal-v1 -> process_refund OK {'refunded': 4200}      <- silent, valid, catastrophic
# the fix, after the audit:
carrier-portal-v1 -> process_refund  PermissionError: carrier-portal-v1 lacks refund:write for process_refund
```

🕒 WHAT TO NOTICE

Four audit lines for four calls — denials included. The third call authenticated perfectly and refunded ₹4,200 to a carrier portal. The fix is one dictionary line because the model was scopes-in-a-table.

Sabotage — break exactly one thing, predict, run, fix it back

Break this	Predict, then observe (real result)	Lesson
line 15: drop the <code>""" not in granted' clause</code>	carrier-portal-v1 is now DENIED even before the fix. The system got SAFER.	A check you did not test both ways is a guess. Notice the feeling; log it.
call <code>run_tool("kavi-agent", "send_sms", {})</code>	KeyError: 'send_sms' — an unlisted door does not open.	REQUIRED is the model; a tool nobody tabled cannot be authorized.
swap lines 20/21: <code>audit_log</code> before <code>authorize</code>	The log line reads ALLOW for the denied call.	Order is the lesson: authorize → audit → tool.

Lab E4 · Chunking three ways on SOP 214

Objective 3.5 · Cut the same document by tape measure and by seams; ask whether the re-filing procedure survives whole.

Run

```
python e4_chunking.py
```

PREDICT FIRST — write it in the notebook

SOP 214 is ~1,330 words, nine headed sections. Fixed size=180: how many boxes? Heading: how many chunks (count the preamble)?

For each strategy: does the whole procedure (Step 1 ... Step 6) land inside ONE chunk?

Which fixed box holds Step 1, and does that box also mention cancellation?

WHAT THE KIT ACTUALLY PRINTS

```
fixed   :   8 chunks | full procedure in one chunk: False
heading :  10 chunks | full procedure in one chunk: True
  fixed box 3: sections starting inside ['3'] | 'e-way bill' x6 | 'cancel' x9 | Step 1 False Step 6
False
  fixed box 4: sections starting inside ['4'] | 'e-way bill' x6 | 'cancel' x3 | Step 1 True Step 6 F
alse
  fixed box 5: sections starting inside ['5'] | 'e-way bill' x3 | 'cancel' x0 | Step 1 False Step 6
True
```

🕒 WHAT TO NOTICE

Box 4 opens with the tail of §3 (three 'cancel's), carries Steps 1-5, and stops before Step 6, which fell into box 5. 'e-way bill' appears six times in BOTH box 3 and box 4 — the retriever cannot tell cancellation from re-filing by vocabulary. The heading cut carries '[SOP 214 - 4. Re-filing...]' inside the chunk text.

Sabotage — break exactly one thing, predict, run, fix it back

Break this	Predict, then observe (real result)	Lesson
size=270	fixed: 5 chunks; procedure whole: True — but each box now spans two sections.	Bigger boxes 'fix' one miss by making every answer 40% off-topic; granularity must match the questions.
regex ^# only (top-level heading)	heading: 1 chunk (the whole document) procedure whole: True	One box per document is 'context is king' and kills precision.
tail=100 in with_overlap	Each chunk now repeats ~40% of its neighbour.	Overlap is a dial; past a few dozen tokens you pay twice and retrieve echoes.

Lab E5 · The hybrid — the register and the toy foreman

Objective 3.6 (and 3.5) · Two queries; predict which half of the hybrid carries each. The DRY_RUN foreman is a toy: bag-of-words + a tiny synonym memory, deliberately BLIND to hyphenated/coded strings (like a fuzzy embedding).

Run

```
$env:DRY_RUN = "1"; python e5_hybrid.py
```

PREDICT FIRST — write it in the notebook

'e-way bill re-filing' — the exact phrase exists in §4. Which half carries it?

'parcel stuck, paperwork problem, what should I do' — no SOP says stuck, parcel or paperwork. Which half rescues it?

WHAT THE KIT ACTUALLY PRINTS

q: e-way bill re-filing

```
['[SOP 214 - 4. Re-filing an e-way bill after a hold]', '[SOP 214 - 3. Cancellation of an e-way bill]', '[SOP 214 - 8. Message templates]']
```

q: parcel stuck, paperwork problem, what should I do

```
['[SOP 214 - 1. What a customs hold is]', '[SOP 214 - 8. Message templates]', '[SOP 214 - 4. Re-filing an e-way bill after a hold]']
```

WHAT TO NOTICE

Both queries land the right section at rank 1 — and only because both halves ran. The kit's bm25ish now has the property the Dock names: rare words count more (IDF), stopwords count nothing.

Sabotage — break exactly one thing, predict, run, fix it back

Break this	Predict, then observe (real result)	Lesson
w_sem=1.0 (kill the register)	q1 flips: §3 Cancellation rises to rank 1 — Padma's incident, reproduced. q2 unchanged.	The exact-phrase query needed the register; the toy foreman is blind to 're-filing'.
w_sem=0.0 (kill the foreman)	q1 unchanged. q2 collapses: the preamble chunk, then §1, then §2 — zero keyword overlap, arbitrary order.	The symptom query needed the foreman; the register had no words to match.
remove 'stuck' from SYN	q2's §1 drops in rank.	The foreman's memory of meaning is the synonym bridge — a real embedding has it built in.

Lab E6 · The retrieval router — three shapes, three machines

Objective 3.6 · Route three questions; see each handler answer from its own world (stubs stand in for the TMS and the database).

Run

```
python e6_shape_router.py
```

PREDICT FIRST — write it in the notebook

Predict all three routes. Then: what would `hybrid_rag` return for 'How many shipments ran late this week?' and why is that worse than an error?

WHAT THE KIT ACTUALLY PRINTS

```
structured_lookup <- Where is KF-2481 right now?
                    -> {'status': 'CUSTOMS_HOLD', 'leg': 'ICD Pune'}
sql_aggregate      <- How many shipments ran late this week?
                    -> 41
hybrid_rag          <- What is the procedure to re-file an e-way bill?
                    -> answer drafted from [SOP 214 - 4. Re-filing an e-way bill after a hold]
```

🕒 WHAT TO NOTICE

The id question returned THE row; the count returned a number the accountant counted; only the procedure went to the shelves. Three lines, Law 3.

Sabotage — break exactly one thing, predict, run, fix it back

Break this	Predict, then observe (real result)	Lesson
route 'How many times has KF-2481 been delayed?'	<code>structured_lookup</code> wins (the id regex fires first) — and returns one row, not a count.	Rule order matters: aggregate language should outrank the id pattern (Dock Book II Q35).
delete the aggregate rule	'How many...' falls to <code>hybrid_rag</code> and 'answers' from an SOP chunk.	A fluent wrong number: nobody counted anything.
route 'Where is my parcel?' (no id)	<code>hybrid_rag</code> — the router cannot know it is a lookup without an id.	Ask for the id; do not guess a status from prose.

Lab E7 · The cameras — one live ticket, then the canned day

Objective 3.4 · Watch one ticket write four JSON lines with a trace id, then digest 40,000 lines into mean and p95 per stage.

Run

```
python gen_logday.py          # once
python e7_dock_cameras.py
```

PREDICT FIRST — write it in the notebook

Which stage will own most of the p95? Will any stage's mean look healthy while its p95 is awful? Write two numbers for 'retrieve'.

WHAT THE KIT ACTUALLY PRINTS

```
# one live ticket through the cameras:
{"trace": "3a523fa3", "stage": "route", "ms": 10, "tokens": 400, "cost_pc": 1}
{"trace": "3a523fa3", "stage": "retrieve", "ms": 20, "tokens": 3600, "cost_pc": 3}
{"trace": "3a523fa3", "stage": "draft", "ms": 30, "tokens": 5800, "cost_pc": 41}
{"trace": "3a523fa3", "stage": "total", "ms": 61}
# the canned day, digested:
route      mean=    93ms p95=   134ms
retrieve   mean=   399ms p95= 2,597ms    <- the mean lied
draft      mean=   879ms p95= 1,207ms
total      mean= 1,396ms p95= 3,543ms
```

WHAT TO NOTICE

6% of retrievals in the canned day hit a cold carrier API (gen_logday.py, seeded). The mean is 399 ms; the p95 is 2.6 s; the total p95 breaches a 3 s SLA while every mean looks fine. Open logday.jsonl and grep one trace id: four lines, one journey.

Sabotage — break exactly one thing, predict, run, fix it back

Break this	Predict, then observe (real result)	Lesson
mint the trace id INSIDE the stage loop	digest() prints identical numbers. grep any id: one line only — the journey is gone.	Observability gaps don't error; they wait.
gen_logday: tail probability 0.06 → 0.00	retrieve p95 ≈ 340 ms; total p95 ≈ 2.0 s.	The tail IS the incident; remove it and the SLA looks safe — which is what the mean was already telling you, wrongly.
log(stage=..., consignment=ticket) and digest by consignment	10,000 series instead of 4.	The cardinality trap, on your own laptop.

Lab E8 · Two bills for the same task

Objective 3.8 · Sum the monolith and the catalogue; then the monolith's honest case.

Run

```
python e8_two_bills.py
```

PREDICT FIRST — write it in the notebook

Sum both bills in the notebook first. At what corpus size and stability does the third line beat the second?

WHAT THE KIT ACTUALLY PRINTS

```
monolithic : 362,460 tokens/call
progressive : 4,205 tokens/call + 1 round-trip
small+cached: 4,500 token-equivalents/call, no extra turn
```

🕒 WHAT TO NOTICE

86× between the first two (the Dock printed 4,625 — see the honesty note; the code's 4,205 is right). The third line is the monolith done RIGHT: small + stable + always-used + cached ≈ progressive cost, zero latency tax.

Sabotage — break exactly one thing, predict, run, fix it back

Break this	Predict, then observe (real result)	Lesson
sop_corpus: 400 * 900 → 45 * 900	monolithic drops to 42,960 — and now fits behind caching at ~4,300 equivalents.	The ≤200k threshold is where the monolith becomes the honest answer.
retrieved: 20 * 180 → 5 * 180	progressive 1,505 — cheaper, and measurably worse (20 beat 5 and 10).	Reading only the token column is advertising, not analysis.
cache_hit_price 0.1 → 1.0	small+cached 45,000 — ten times the progressive bill.	Without the cache discount the monolith's case collapses; caching minimums are why the tier matters.

A surprise is any place where your prediction and the printed result disagree. That gap is the entire value of the kit; write it in your own words.

[illegible]

1. Every expected output in this manual was produced by running the kit; if yours differs, diff your hand-typed file against the kit's.
2. DRY_RUN is the bench. E2's live count_tokens is free; E5's real_embed is where money starts.
3. The kit's SOP 214 is real enough to fail the way Padma's did: box 4 holds Steps 1-5 and the tail of cancellation.
4. Sabotage that makes the system SAFER (E3) is still sabotage: you learned the check's shape.
5. The Dock's 4,625 is a slip; the code's 4,205 is right. Trust what runs, and say so in the DRIFT voice.